Overlapping Ring Neighbor Monitoring Algorithm

Jon Paul Maloy, M. Sc.

Ericsson Canada Inc
Montréal, Canada
jon. maloy@ericsson.com

Abstract

Keeping track of neighboring nodes' availability inside a cluster has always been a resource demanding task, and this demand tends to grow more than linearly with increasing cluster size. This paper presents an algorithm that makes it possible for each cluster member to monitor the presence of more than a thousand neighbor nodes without requiring unreasonable amounts of CPU and network resources, while still being able to discover failure of any of those nodes within a second. The algorithm is combining the best features of the ring monitoring, Gossip and TIPC neighbor monitoring protocols, while largely avoiding the drawbacks of those. The described algorithm was introduced into TIPC in Linux 4.7. It should be noted that this paper deals with the problem of neighbor loss detection only; neighbor discovery is assumed to be handled by a different algorithm

Keywords

Ring Monitoring, Gossip, TIPC, Neighbor Monitoring, Neighbor Supervision.

Introduction

There are at least two reasons for wanting to monitor the availability of cluster neighbors. The first one is to have established connections to a failing node aborted within a short time range, preferably a second or less, instead of waiting for minutes or even hours as is currently often the case. The second is to provide a fast and generic availability service to subscribing users, such as cluster managing software or consensus protocols.

The traditional solution to the first task has been to speed up the connection keepalive timer to sub-second level. But this approach is unsuitable as a general solution, because a cluster and even a single node might host thousands of simultaneous connections, with each endpoint requiring a high-speed timer. As CPU load increases linearly with number of peer sockets to monitor, and network load increases with the square of that number, the resource demand would quickly become unsustainable. Experience shows that it is impractical to have much more than a hundred such connections to monitor per node.

The second task is typically solved by keeping a set of daemons, one on each node, and let those maintain a full mesh of tightly monitored connections dedicated to discovering node failures. This approach provides the desired service, but has several weaknesses. If a daemon crashes or starves while the rest of the node is working well, users will receive false positives, potentially leading to inconsistent cluster views and confusion. Neither does this service provide much help with solving the first problem, as there is no built-in association between the monitored connections and all the other connections to the same node. Finally, this solution suffers from the same scalability limitation as the first approach; it cannot scale much beyond one hundred nodes without causing unreasonable CPU and network load.

It would be highly desirable if we could achieve a service of the second type which also provides a built-in solution to the first problem, while at the same time scales at least an order of magnitude better than current full-mesh approaches. The algorithm described in this paper fulfils these requirements.

State of The Art

There are currently at least three algorithm types to consider when looking for a solution; full-mesh monitoring, ring monitoring and Gossip type protocols. We describe and exemplify each and one of them in this chapter.

TIPC Neighbor Monitoring before Linux 4.7

TIPC has from the beginning provided a solution to both the first and the second tasks. [1][2] To achieve this, each node maintains one or two actively monitored links to all neighbor nodes, as is shown in fig. 1 below. Each link endpoint in their turn keep a list of references to all local sockets having a connection to its peer node. When connectivity to a node is lost all associated sockets can hence be notified via a dedicated pseudo message telling them to break the connection. At the same time, a generic service registry is informed about the event, so that other interested users also may receive notifications.

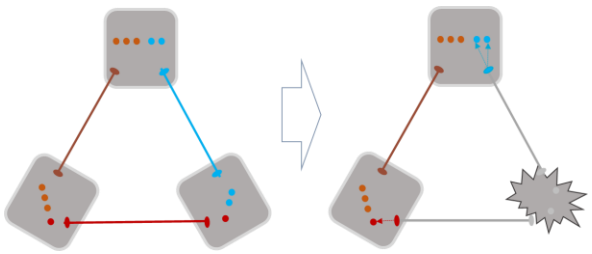


Fig 1: Aborting connections when associated link is broken

While the first part of the requirements stated in the introduction chapter is fully satisfied, the second part, about scalability, is not. Since the actively monitored links are set up in a full mesh pattern, CPU load will increase by O(N),

while network load grows by $O(N^2)$. We need a better solution to this if we want to get beyond the one hundred node barrier.

Ring Monitoring

A possible alternative to the full-mesh scheme to use a ring topology, as depicted in fig. 2. The nodes are sorted into a logical ring based on some common criteria, and each node takes on the task of monitoring the next downstream neighbor in the list. Node failures can then be detected in different ways, e.g., by detecting the loss of a circularly roaming token, as is done by the Totem Single-Ring Protocol - the primary configurable alternative in Corosync, or by heartbeats – the second alternative. [3] [4]

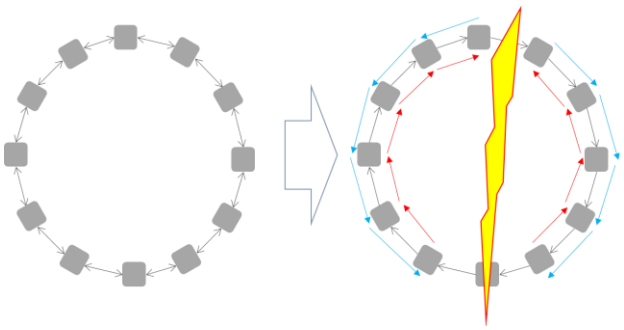


Fig 2: Ring Neighbor Monitoring

This scheme has the advantage that it is light-weight and provide fast failure discovery when everything is working smoothly and the cluster is reasonably small, i.e., a few dozens of nodes.

But there are also disadvantages. In a token based system, failure discovery time increases by the size of the cluster, because of the time that must be granted to the token to iterate the ring. Another disadvantage is that the task of diagnosing what happened, e.g., which node really failed, or if there was an accidental network partition, is complex and resource consuming. The same is true for the recovery actions to be taken to reach a new consensus. In both cases significant amounts of internal messaging, typically based on UDP multicast, is needed to make all nodes agree on a new ring configuration. Finally, keeping any amount of shared state across many nodes in a large and dynamic cluster is generally undesirable and should be avoided.

The Gossip Protocol

Gossip is a class of protocols which is typically used in large networks of nodes, most notably by the BitTorrent client Tribler. [5][6] It is a well-proven tool for solving the scalability problem. In this scheme, each node has direct knowledge about and monitors only a small and randomly selected subset of its neighbors. The nodes distribute information, such as their network view, at regular intervals to their known neighbors, which in their turn will spread the information further. After some generations, the information will reach even the farthest-away nodes in the network. In

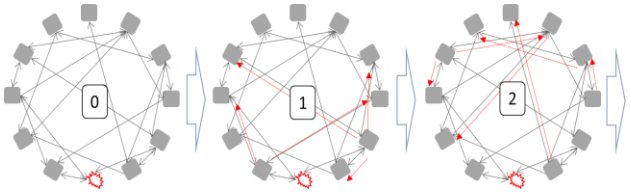


Fig. 3: Gossip information propagation

fig. 3 above we show an example including the first two generations of node loss information propagation.

This scheme is both powerful, simple, and robust, and scales extremely well, but it also has drawbacks. The most notable is the randomness of the information propagation, leading to non-deterministic and potentially very long intervals before all nodes are informed about a change. The periodic, multi-hop nature of the information exchange aggravates this problem. A second problem with Gossip is that this type of information spreading inevitably leads to information duplication. A node may easily receive the same, hopefully consistent, information multiple times from different sources.

While being acceptable in loosely connected node groups of BitTorrent hosts, both the duplication and long and unpredictable loss detection times are undesirable in the more tightly connected type of clusters used in HA systems.

The Challenge

When analyzing the three protocols described above, we see that an optimal solution to our stated wish list would be to find an algorithm which maintains the connection-service registry-link hierarchy of TIPC, inclusive the full-mesh link connectivity, but avoids its full-mesh active link monitoring. Ring monitoring offers a light-weight monitoring scheme that we could leverage, but we must avoid its statefulness and vulnerability to network partitioning. Likewise, Gossip offers scalability and simplicity, while we want to avoid its long and non-deterministic error propagation times.

Overlapping Ring Monitoring

The Overlapping Ring Monitoring algorithm, depicted below in fig. 4, offers a response to this challenge. We retain the TIPC full-mesh link topology, but by combining some features of ring supervision with some others from Gossip, we avoid the $O(N^2)$ monitoring signaling growth from the earlier protocol version.

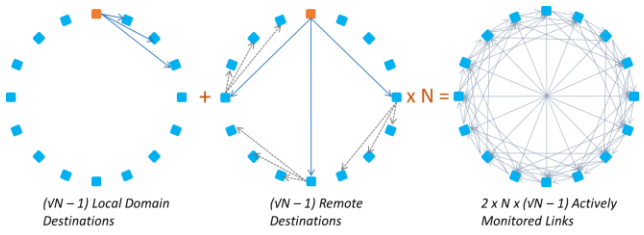


Fig. 4: Overlapping Ring Monitoring algorithm

Just like in other ring protocols, the nodes are organized into a ring. However, in contrast to present state of the art, we don't make any attempt to reach a consensus among the nodes about this topology. Instead, each node defines its own ring, only based on its own direct discoveries. The size and memberships of each ring may hence differ, but since the criteria for ordering the members are the same on all nodes, they will never be contradictory.

Based on its own ring, plus information received directly from the ring's member nodes, each node can now go ahead and define its own Gossip-style monitoring topology, but without its randomness and multi-hop information propagation. The algorithm is described more in detail below.

- Each node sorts all its N directly discovered neighbors into a circular list. In TIPC this is based on the numerical value of the node identity.
- The node defines the next (√N – 1) nodes downstream from itself in the list as its "local monitoring domain," and supervises those nodes actively.
- It distributes a record describing its monitoring domain to all the neighbors. A new record is distributed whenever there is a change in the local domain, i.e., a node was added or removed, or the link to it was lost or re-established.
- When it receives a domain record from a node outside its domain, it stores its information and uses it to keep track of the status of those nodes.
- In order to handle failures causing a partitioned network, the node also selects a set of members outside its local domain to monitor actively. Those nodes are selected so that no members of its ring is more than two active supervision hops away. In a full-mesh network there will be (√N – 1) such nodes, which we have chosen to call remote domain "heads". This guarantees that the node will discover loss of indirectly monitored ring members in a remote partition within a period marginally longer – the time to perform one hop – than the detection time for actively monitored nodes.
- The node will immediately and autonomously re-calculate a new optimal monitoring topology upon any detected change in its ring – be it self-discovered or based on a received domain record. Note that calculating the new monitoring topology is an entirely local matter, and involves no additional external messaging.

Loss of Local Domain Member

When a node discovers a change in its local domain, e.g., finds that a member has become unresponsive, it is its responsibility to send out an updated "domain record" to all other nodes in the ring informing about the change.

For convenience, and because domain records anyway are small (140 bytes in a 1000-node cluster,) we re-send the whole record at every change. This makes it easier for the receivers to just re-apply it to their local ring, using the same algorithm every time. A domain record, as shown in fig. 5, contains the following data.

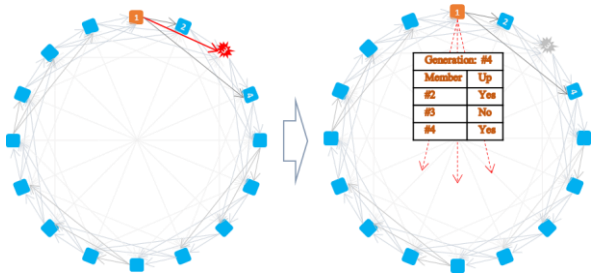


Fig. 5: Loss of local domain member

- An array containing the node identities of the domain members, in ascending numerical order. In TIPC, this identity is a four-byte integer.
- The availability status of those members. In TIPC, this has been implemented as a bit-array, where each bit's position matches the position of the corresponding member in the member array.
- A domain generation number. This number is stepped each time there is a change in the domain, and makes it easy for receivers to identify and ignore duplicate records. The latter may happen because the records may need to be re-transmitted, as will be explained further down in this paper.

Loss of Actively Monitored Remote Member

When a node discovers the loss of an actively monitored ring member outside its own domain – a remote domain head – it must do the following.

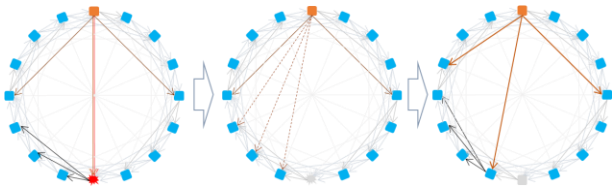


Fig 6: Loss of actively monitored "head" member

- It must send at least one "confirmation probe" to the lost node's domain members, to verify that they are still available, i.e., that the problem we have at hand isn't a partitioned network. Once those members respond – which they must do immediately – the probing is stopped. Otherwise it continues probing until even those member(s) are considered lost.
- It must re-calculate the monitoring topology, taking into account the lost member(s). Typically, this means that the "head member" indexes are moved forward by one step for the remaining part of the ring, as shown in fig. 6 above.

Loss of Indirectly Monitored Member

A member being defined to be outside a node's local domain, but reported to belong to an actively monitored remote member's domain, does not need to be actively supervised. We call such members "indirectly" monitored members. It is important to note that this designation, just like the designation "head", is valid only locally on the designating node, and only for the moment, for its currently applied topology.

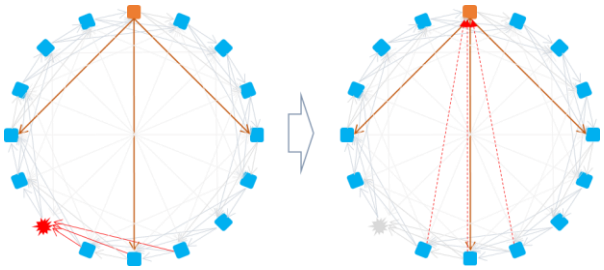


Fig. 7: Loss of indirectly monitored member

When such a member crashes or is disconnected, there will normally be (√N – 1) other members actively monitoring it. Immediately upon the detected loss, those members will send out updated domain records to the whole cluster. This means that any node viewing the member as "indirectly" monitored most often need only to observe the incoming events to determine the member's status. The current algorithm is to initiate confirmation probing at the first received "down" event, and consider the node lost after four such events, unless the probing has proven the opposite. This algorithm entails a theoretical risk of false positives, but those should normally have no catastrophic effect. In the extremely unlikely case that the member might still be up, the discovery protocol would soon restore connectivity.

Differing Cluster Views

For pedagogical reasons, the examples presented so far in this paper have been based on a symmetric, full-mesh cluster which size happened to be a square number. This is not the daily reality of most cluster topologies, and definitely not so during transient situations, when multiple nodes may be coming up or going down simultaneously.

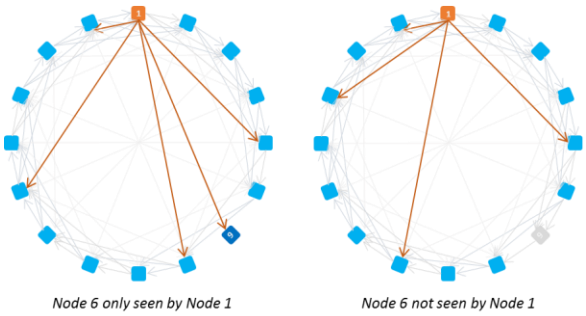


Fig. 8: Two nodes with different views of the cluster

Despite this, a thorough analysis reveals that there are still only two seemingly "special" cases to consider, and that those in reality are both fully normal and frequently occurring situations. The two cases are illustrated in fig. 8.

- The first case is when a node discovers a neighbor which no other member has reported as part of its domain. In this case, the standalone neighbor must be sorted into the ring just like any other member.

  Since nobody else is monitoring the new member actively, the discovering node must take that task on itself. Depending on the new member's position in the ring, it will be defined either as a member of the local domain or as a new remote domain head, the latter independently of whether it reports a domain or not.

  If the new neighbor reports to have a local domain, the domain members which are already known by the discovering node are "applied" to its monitoring topology during the following re-calculation, fully in line with the algorithm. If a remote domain member node is yet undetected by the discovering node, it becomes subject to treatment in accordance with the second case, which is described below.

  The topology re-calculation algorithm must still take special care in this case. If the preceding domain head in the ring reports a domain with members which are placed beyond the singleton member's position, that domain is truncated. I.e., the fact that those members are monitored by the preceding head node is ignored.
- The second special case might be regarded as the opposite to the first one. If a known node reports a domain with members which are yet undiscovered by the receiving node, those members are not added to its ring. Still, the knowledge about those members are kept for potential future use, and they remain listed as "non-applied" in the node's topology structure.

Properties

The algorithm we just have described is fully auto-adaptive and requires no configuration input whatsoever. Apart from that, it has several nice properties as compared to the three algorithms we referred to initially.

It retains the hierarchical nature of the legacy TIPC algorithm, but CPU load now increases by O(√N) instead of as previously O(N), while network load now grows by O(N√N) instead of by O(N^2). To put this into perspective: while each node in a 65-node cluster earlier had to actively monitor 64 links, that number is now sufficient to sustain full monitoring of each neighbor in a 1000-node cluster. When it comes to network load, the corresponding figures may look less impressive; - 4096 actively monitored links increases to 64,000. Still, assuming an average background load of 1000/375 x 60-byte probe messages per second per link – corresponding to a total extra load of 82 MB/s through the communication backplane - the amount of extra traffic is easily sustainable for any modern switch fabric.

If we compare with the typical ring monitoring protocol we will notice that the new protocol maintains no shared state, as we don't need to reach any consensus about the network topology, and don't assign special roles to any of the nodes. A node defines its topology solely based on its own discoveries and on information received directly from its neighbors, making no assumptions about the other nodes' knowledge about itself or about the overall cluster topology. Because of this, it can now swiftly and independently adapt to topology changes without consulting anybody else. Furthermore, since no cluster node now is more than two active monitoring hops away from any other node, the risk of an undetected accidental network partitioning is eliminated.

Comparing the new algorithm to the Gossip algorithm, we see that we have eliminated the stochastic element of the latter. Any information propagation now performs maximum one network hop, making failure discovery times highly predictable. To summarize the latter; if active monitoring failure detection time is T seconds, maximum indirect failure discovery time will be T + T/4 seconds. The latter is because information propagation is driven by a per-link timer, which is set to be 1/4 of the configured failure discovery tolerance. In the extreme case of a network partition, it might still take up to 2 x T seconds to discover loss of indirectly monitored nodes inside the lost partition. Assuming the default link tolerance of 1.5 seconds, we hence get worst case failure detection times of 1.5 seconds, 1.875 seconds, and 3.0 seconds, respectively. It should however be noted that the configured tolerance easily can be reduced to sub-second levels.

Implementation and Scaling

One critical aspect of the implementation was how to distribute domain records, as those are sent to all ring members at any change in the local domain. In the choice between multicast and periodic unicast updates we opted for the latter. Domain records are therefore piggybacked on link state/probe messages, which are driven by a timer expiring each 375 ms. In accordance with the new algorithm, the sending of such messages can most often be suppressed, but obviously that is not the case when there is a change in local domain. Since link state messages are sent in a best-effort manner, we had to add the already mentioned generation counter to the domain record.

Regarding scalability, we still haven't attempted to set up a 1000-node cluster, mostly due to lack of time and available HW resources. Our largest setup so far consisted of 800 nodes running in a stable topology.

The algorithm we have described added ca 500 lines of C code to the already existing code base in Linux 4.7.

Potential for Improvements

While the described algorithm defines a two-level neighbor monitoring hierarchy, there is in theory nothing stopping us from defining a hierarchy of three levels or even more. The reward would be that each node now would monitor only ($3x^3\sqrt{N}$) neighbors etc., but the implications regarding complexity and possible increased fragility remain to be analyzed.

Another possible improvement would be to reduce the number of running timers. With the current implementation, we have one timer for each link, expiring every 375 ms. When it expires it will most often find that there is nothing to do, and go right back to sleep. Still, the high number of running timers will cause unnecessary CPU load that almost certainly could be reduced.

Conclusion

We have achieved an algorithm which significantly improves the ratio size/failure detection time in large clusters. Because of this, it will now be easier than before to scale up clusters to hundreds of nodes with no negative impact on performance and stability. By applying the algorithms to TIPC we have made it an even better base than before for developing new services in HA clusters and cloud environments.

Acknowledgements

Thanks to my colleagues in Ericsson who have been reviewing and commenting the solution. Special thanks to my fellow TIPC code maintainers Richard Alpe and Parthasarathy Bhuvaragan from Ericsson and Xue Ying from WindRiver for reviewing the code and suggesting improvements before delivery.

References

[1] J. Maloy, "Transparent Inter Process Communication" https://www.slideshare.net/JonMaloy/intro-to-the-tipc-messaging-service

[2] J. Maloy, A. Stephens, "TIPC: Transparent Inter Process Communication" http://tipc.sourceforge.net/doc/draft-spec-tipc-10.html

[3] Y. Amir, L.E. Moser, P.M. Melliar-Smith, D.A. Agarwal, P. Ciarfella: "The Totem single-ring ordering and membership protocol", Univ. of California, Santa Barbara, 1995

[4] "The Corosync Cluster Engine" http://corosync.github.io/corosync/

[5] G. R. Subramaniyan, P. Raman, A. D. George, M. Radlinski, "GEMS: Gossip-Enabled Monitoring Service for Scalable Heterogeneous Distributed Systems", Univ. of Florida, 2006

[6] Tribler, https://github.com/Tribler/tribler/wiki

Author Biography

Jon Maloy is an employee of Ericsson, with a past both at its Stockholm and Montreal sites. His main activities have been in the OS and HA Cluster areas, with special focus on networking and inter process communication. He is the originator and principal maintainer of the TIPC inter process communication service in Linux.